



SMART CONTRACT SECURITY ASSESSMENT

Up

SECURITY REVIEW

21 JULY 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Up	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	4
5.2	Scope	5
6	Findings Summary	5
7	Findings	6

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Up

Up. is a ve(3,3) decentralized exchange on Robinhood Chain. It pays the people who make a market work, and it lets them decide where that pay goes.

Every exchange needs three things: traders who bring volume, liquidity providers who make trades cheap, and someone to decide which markets deserve support. up. assigns each job a clear reward. Traders get routed execution across two pool designs. Liquidity providers earn UP emissions when they stake in gauges. And people who lock UP into veUP collect 100% of the protocol's trading fees in exchange for steering emissions with their votes.

The protocol runs on a weekly epoch rhythm. Each epoch, veUP holders vote on which pools should receive the coming epoch's UP emissions. Emissions attract liquidity; liquidity tightens prices; tighter prices win volume; volume generates fees; and those fees flow back to the voters who directed the emissions in the first place.

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** - almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** - still relatively likely, although only conditionally possible
- **Low** - requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 5 days with a total of 80 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying three Medium and three Low severity issues. They're mainly related to insufficient enforcement of protocol invariants across permissionless and governance-controlled execution paths.

The Up team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Up
Repository 1	up-contracts
Repository 2	up-slipstream

Type of Project	ve(3,3) DEX
Security Review Timeline	5 days
Review Commit Hash 1	1e5ebbce0a8ae5904224beec6a9fadc7e0fafa89
Review Commit Hash 2	647f599031d5333ca5d000b57fe9b26ede0dc14a

5.2 Scope

The changes in the following repos were in the scope of the security review:

FOLDERS
up-contracts
up-slipstream

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **3**
- **Low** issues: **3**
- **Info** issues: **4**

ID	TITLE	SEVERITY	STATUS
M-01	Permissionless <code>poke()</code> Allows Anyone to Zero Out a Decayed-Lock veNFT's Current-epoch Bribes/Fees	MEDIUM	ACKNOWLEDGED
M-02	Invalidating a Locked Report Can Trigger Uncapped ("Fail-Open") Emission Release	MEDIUM	ACKNOWLEDGED
M-03	Enforced-mode Gauge-cap Enforcement Can Be Bypassed Via Permissionless <code>updateFor</code>	MEDIUM	ACKNOWLEDGED
L-01	Provenance Fields Can Be Silently Rewritten on Locked Reports	LOW	ACKNOWLEDGED
L-02	<code>expiredIndexFloor</code> Strands a Gauge's Emission Share Whenever a Full Epoch Is Missed, in Every Cap Mode	LOW	ACKNOWLEDGED
L-03	<code>governanceCapActive</code> Unconditionally Disables the <code>emergencyCouncil</code> Circuit Breaker in <code>_distributeEnforced()</code>	LOW	ACKNOWLEDGED
I-01	No Upper Bound on Emission Multiplier	INFO	ACKNOWLEDGED

ID	TITLE	SEVERITY	STATUS
I-02	CLTwapOracle.sol and <code>Gauge._pendingFees()</code> Are Unreferenced Dead Code	INFO	ACKNOWLEDGED
I-03	<code>Voter.reset()</code> Authorizes Against Raw <code>msg.sender</code> Instead of <code>_msgSender()</code> , Breaking Meta-tx Calls	INFO	ACKNOWLEDGED
I-04	Unescaped Token Symbols in On-Chain SVG Allow XML Injection and Broken NFT Metadata	INFO	FIXED

7 Findings

M-01

MEDIUM

ACKNOWLEDGED

Permissionless `poke()` Allows Anyone to Zero Out a Decayed-Lock veNFT's Current-epoch Bribes/Fees

Description

The `poke()` is meant to let a veNFT holder (or an approved operator) refresh/re-checkpoint their own vote weight without changing their pool selection. The new implementation adds a special case in `_poke` for when `balanceOfNFT(tokenId)` has decayed to 0: instead of going through `_vote` (which enforces `ZeroBalance` revert protection), it calls `_reset(tokenId)` directly and returns.

The `_reset()` is an internal helper with no access control of its own, the only access control for resetting a tokenId's votes normally comes from the external `reset()` function's `onlyNewEpoch()` modifier and `isApprovedOrOwner()` check (contracts/Voter.sol:308-311). The new zero-weight branch in `_poke` bypasses both of these protections, since `poke()` itself only gates on `block.timestamp` vs. `epochVoteStart()` (contracts/Voter.sol:340) and never checks ownership/approval or epoch-timing restrictions tied to the token.

Attack flow:

1. A veNFT's lock decays such that `balanceOfNFT(tokenId) == 0` while it still holds votes/checkpoints for the current epoch (weight can hit 0 before the epoch's rewards have been fully claimed/settled).
2. Any address calls `poke(tokenId)` after `epochVoteStart`.
3. `_weight == 0` → `_poke` calls `_reset(tokenId)` directly, bypassing `isApprovedOrOwner()` and `onlyNewEpoch()`.
4. `_reset()` calls `IReward(gaugeToFees[gauge])._withdraw(votes, tokenId)` and `IReward(gaugeToBribe[gauge])._withdraw(votes, tokenId)` for each pool the victim voted in.

5. `Reward_withdraw()` → `_writeCheckpoint(tokenId, balanceOf[tokenId])` overwrites the *existing* current-epoch checkpoint (since `epochStart` of the last checkpoint equals `epochStart` of `block.timestamp`) with a balance of 0, rather than appending a new checkpoint.
6. The victim's `earned()` for that epoch is computed against the now-zeroed checkpoint, destroying their claim to that epoch's bribes/fees. The pool's total supply checkpoint is unaffected by this call, so the victim's forfeited share is effectively redistributed proportionally to all other depositors in that Reward contract for the epoch.

No caller-side incentive or profit is required for the attacker, this is griefing that harms the victim and benefits arbitrary third parties, and can be automated to grief every decayed-lock veNFT each epoch.

Location of Affected Code

File: [contracts/Voter.sol#L338-L343](#)

```
function poke(uint256 _tokenId) external nonReentrant {  
    if (block.timestamp <= ProtocolTimeLibrary.epochVoteStart(block.timestamp)) revert DistributeWindow();  
    uint256 _weight = IVotingEscrow(ve).balanceOfNFT[_tokenId];  
    _poke(_tokenId, _weight);  
}
```

Impact

Loss of funds / unfair reward redistribution.

Recommendation

Require ownership/approval before allowing the zero-weight cleanup path to execute, matching the protection already present on `reset()`.

Team Response

Acknowledged.

M-02

MEDIUM

ACKNOWLEDGED

Invalidating a Locked Report Can Trigger Uncapped ("Fail-Open") Emission Release

Description

`publishGaugeEpochValue()` explicitly blocks any update to a locked report that would expand its allowed emission capacity (`_reportExpandsCapacity` check). However, `invalidateGaugeEpochReport()` has no equivalent locked check, governor or emergencyCouncil can flip the status of an already-locked, already-used report to `REPORT_STATUS_INVALID` at any time, with no restriction based on lock state.

In `Voter_capFallbackAllowed()`, any non-`VALID` status is treated identically to "no report exists," and is routed into the fallback-cap chain (emergency cap → default cap → global fallback cap). Since `defaultGaugeCapActive` and `globalFallbackCapActive` both default to false and must be explicitly configured by governance, an invalidation performed on a locked report, with no fallback cap active, causes any subsequent `quoteGaugeCap` call in the same epoch to fail open, allowing release of up to 100% of the requested cumulative emission, uncapped.

Location of Affected Code

File: [contracts/gauge-caps/GaugeCapController.sol#L224-L229](#)

```
function invalidateGaugeEpochReport(address _gauge, uint256 _releaseEpoch) external onlyAuthorizedInvalidator {
    StoredReport storage stored = reports[_gauge][_releaseEpoch];
    if (stored.reportVersion == 0) revert InvalidReport();
    stored.status = REPORT_STATUS_INVALID;
    emit GaugeValueReportInvalidated(_gauge, _releaseEpoch, msg.sender);
}
```

Impact

An action intended to flag untrustworthy data ("stop trusting this report") can produce the functional opposite of its intent: removing the cap entirely rather than restricting it. If a second distribution pass or partial release occurs in the same epoch after invalidation, a gauge can receive uncapped emissions instead of a reduced or blocked amount, directly undermining the purpose of the cap system.

Recommendation

Block invalidation of locked reports entirely (`if (stored.locked) revert InvalidReport();`), forcing corrections through the existing `_reportExpandsCapacity-gated` republish path instead.

Team Response

Acknowledged.

M-03

MEDIUM

ACKNOWLEDGED

Enforced-mode Gauge-cap Enforcement Can Be Bypassed Via Permissionless updateFor

Description

The contract tracks a gauge's accrued-but-uncollected reward index in one internal function, and that function decides which of two payout buckets to credit the accrual into based on whatever the current enforcement mode happens to be in storage at that exact moment. One of those buckets is subject to the enforced cap system (oracle checks, governance caps, emergency caps, fallback policy); the other is paid out later in full, with no cap check of any kind, regardless of what the enforcement mode is by the time it's actually paid.

The problem is that this internal function is the only place that decides which bucket an accrual goes into, but it is not one of the functions responsible for keeping the enforcement mode up to date. Several other entry points do refresh the enforcement mode before doing any accounting, but this particular function, and everything that calls it directly, does not. That includes a set of fully public, unrestricted functions with no access control, as well as the normal voting-related actions (voting, poking, resetting a vote, and moving a position into or out of a managed vault), all of which call into this same accrual function as a side effect of their own logic.

Governance can queue a switch into enforced mode to take effect starting immediately in the current epoch. Until some other function actually applies that queued switch, the enforcement mode stored on-chain still reflects the old, unenforced setting. In that window, anyone can call one of the public, access-control-free entry points (or simply vote/poke/reset on any position touching the target gauge) to trigger the accrual function while it's still reading the stale, unenforced mode. Any backlog of unsynced rewards the gauge was carrying gets credited into the always-pays-out-in-full bucket instead of the capped one, and once it's there, it is permanently exempt from the cap system that governance just turned on, no matter what the enforcement mode becomes afterwards.

Because the vulnerable entry points require no special permission and the exploit is purely a matter of transaction timing at the epoch boundary, this can be executed by anyone. It can also be set up in advance: deliberately avoiding syncing a chosen gauge for one or more epochs before a known enforced-mode activation lets an attacker accumulate a larger backlog, then flush all of it into the uncapped bucket the instant the new epoch begins.

Location of Affected Code

File: [contracts/Voter.sol#L722-L725](#)

```
function _distribute(address _gauge) internal {
    _settleExpiredEpochs();
    _applyQueuedCapModeFor(ProtocolTimeLibrary.epochStart(block.timestamp));
    _updateFor(_gauge);
    // code
}
```

Impact

Anyone can front-run a governance-queued switch into enforced mode by calling the permissionless `updateFor` (or `voting/poking/resetting`) in the window before the mode refreshes, permanently routing a gauge's backlog into the uncapped payout bucket and fully bypassing the emission cap.

Recommendation

Have the accrual function apply any pending, due enforcement-mode change for the current epoch before it reads the mode to decide which bucket to use, the same refresh step that the other, already-correct entry points perform. This closes the timing window entirely, since no code path would then be able to observe a stale enforcement mode when deciding how to bucket a gauge's accrual.

Team Response

Acknowledged.

L-01

● LOW

ACKNOWLEDGED

Provenance Fields Can Be Silently Rewritten on Locked Reports

Description

The "does this update expand capacity" check only inspects **feeValueWeth**, **bribeValueWeth**, and **conservativeUpPriceWeth**. It does not consider `sourceEpoch`, `sourceBlockNumber`, `sourceBlockHash`, `routeConfigHash`, or `sourceDataHash`, all of which are provenance/audit-trail fields establishing what dataset and block a report's figures came from. Since **publishGaugeEpochValue** unconditionally overwrites all stored fields (including these) whenever the capacity check passes, an authorized publisher can rewrite a locked report's provenance data, pointing it at an entirely different source block or dataset, as long as the resulting emission-relevant values don't increase.

Location of Affected Code

File: [contracts/gauge-caps/GaugeCapController.sol#L340-L352](#)

```
function _reportExpandsCapacity(
  StoredReport storage stored,
  GaugeEpochValueReport calldata _report
) internal view returns (bool) {
  if (_report.status != REPORT_STATUS_VALID) return false;
  if (stored.status != REPORT_STATUS_VALID) return true;
  uint256 oldDenominator = stored.feeValueWeth + stored.bribeValueWeth;
  uint256 newDenominator = _report.feeValueWeth + _report.bribeValueWeth;
  if (newDenominator > oldDenominator) return true;
  if (stored.conservativeUpPriceWeth == 0) return true;
  return _report.conservativeUpPriceWeth < stored.conservativeUpPriceWeth;
}
```

Recommendation

Extend `_reportExpandsCapacity()` (or add a parallel check) to also freeze provenance fields once `stored.locked == true`, i.e., disallow changes to `sourceEpoch`, `sourceBlockHash`, `routeConfigHash`, and `sourceDataHash` after locking, independent of whether the financial values are being corrected downward.

Team Response

Acknowledged.

L-02

● LOW

ACKNOWLEDGED

`expiredIndexFloor` **Strands a Gauge's Emission Share Whenever a Full Epoch Is Missed, in Every Cap Mode**

Description

The `_settleExpiredEpochs()` (`contracts/Voter.sol:924-952`) rolls `activeCapEpoch` forward on every epoch boundary and unconditionally sets:

```
expiredIndexFloor = index;
```

at `contracts/Voter.sol:943`, regardless of `capMode`.

The `_updateFor()` (`contracts/Voter.sol:662-694`) then computes the gauge's starting index as:

```
uint256 _supplyIndex = Math.max[supplyIndex[_gauge], expiredIndexFloor];
```

at `contracts/Voter.sol:672`, also unconditionally, regardless of `capMode`.

Pre-diff (base Aerodrome/Velodrome semantics), `_updateFor()` used the plain `supplyIndex[_gauge]` with no floor, so a gauge that wasn't touched for a while simply carried forward its full uncollected accrual, nothing was ever lost, only deferred.

With the floor in place, if a gauge is not touched by `_updateFor()` / `distribute()` at any point within a given epoch, the next epoch rollover moves `expiredIndexFloor` past that gauge's true `supplyIndex[_gauge]`. The next time `_updateFor` runs for that gauge, `_supplyIndex` is clamped up to `expiredIndexFloor`, so `_delta = _index - _supplyIndex` silently excludes the entire index range that accrued during the missed epoch. The corresponding `_share` for that interval is **never computed at all**, it is not credited to `nonEnforcedClaimable`, not credited to `enforcedClaimableByEpoch`, and not refunded to `minter`.

In `CapMode.Enforced`, the matching token amount happens to still get burned, because `notifyRewardAmount` credits `epochReserve[epoch]` independently of any per-gauge bookkeeping, and `_settleExpiredEpochs()` burns the full `epochReserve[expiredEpoch]` regardless of which gauges' shares were actually attributed. So in `Enforced` mode this is (at most) an accounting/attribution inconsistency, not a fund-loss bug.

In the launch-default `Disabled` / `ObserveOnly` modes, however, `epochReserve` is **never funded**, `notifyRewardAmount()` only does `epochReserve[epoch] += _amount` when `capMode == CapMode.Enforced` (`contracts/Voter.sol:620-621`). There is no reserve bucket for these modes to fall back on. The underlying `UP` tokens were already pulled into the Voter contract's balance via `safeTransferFrom()` in `notifyRewardAmount()` (`contracts/Voter.sol:615`), but the index-delta that would have routed them to `nonEnforcedClaimable` (for later claim) or to `minter` (for the dead-gauge refund branch) is discarded by the floor before it's ever computed. The tokens are not distributed, not refunded, and not burned, they simply sit in the Voter contract's balance, permanently unclaimable by anyone.

Location of Affected Code

File: [contracts/Voter.sol#L924-L952](#)

```
function _settleExpiredEpochs() internal {
    uint256 currentEpoch = ProtocolTimeLibrary.epochStart(block.timestamp);
    if (activeCapEpoch == 0) {
        activeCapEpoch = currentEpoch;
        activeEpochStartIndex = index;
        _applyQueuedCapModeFor(currentEpoch);
        return;
    }
    if (activeCapEpoch >= currentEpoch) {
        _applyQueuedCapModeFor(currentEpoch);
        return;
    }

    uint256 expiredEpoch = activeCapEpoch;
    uint256 burnAmount = epochReserve[expiredEpoch];
    epochReserve[expiredEpoch] = 0;
    epochBurned[expiredEpoch] += burnAmount;
    epochSettled[expiredEpoch] = true;
    if (burnAmount != 0) liveAccountedReserve -= burnAmount;
    expiredIndexFloor = index;
```

```

activeCapEpoch = currentEpoch;
activeEpochStartIndex = index;
_applyQueuedCapModeFor(currentEpoch);

if (burnAmount != 0) {
    IUp(rewardToken).burnFrom(address(this), burnAmount);
}
emit EpochCapSettled(expiredEpoch, expiredIndexFloor, burnAmount, liveAccountedReserve);
}

```

Impact

Under `Disabled` / `ObserveOnly` mode (the modes the protocol launches in), any gauge that goes a full epoch without being touched by `_updateFor()` / `distribute()` / `vote()` / `reset()` / `poke()` loses its entire pro-rata emission share for that epoch. The tokens become permanently stuck in the `Voter` contract with no code path to recover them.

Recommendation

Only apply the expired-index floor when it is actually backed by a burn mechanism, i.e. in `Enforced` mode. For all other modes, fall back to the original unfloored behavior so a gauge's uncollected accrual simply carries forward instead of being discarded.

Team Response

Acknowledged.

L-03

● LOW

ACKNOWLEDGED

`governanceCapActive` **Unconditionally Disables the**
`emergencyCouncil` **Circuit Breaker in** `_distributeEnforced()`

Description

The enforced-cap release logic is meant to have two independently triggerable safety mechanisms: a governance-set cap and an emergencyCouncil-set emergency cap, the latter existing specifically as a fast, independent circuit breaker. In practice, whenever a governance cap is marked active for a given gauge and epoch, the emergency cap is never consulted at all, not in the primary branch that decides the release percentage, and not in the secondary clamp that's supposed to additionally restrict the result. Both paths are structured so that the presence of an active governance cap fully bypasses the emergency-cap check, rather than the two caps being combined (e.g., taking the stricter of the two).

There is no separate function that lets emergencyCouncil clear or override a governance cap entry, only the governor can write to that state. So if a governance cap is on the books for a specific gauge and epoch (which can persist indefinitely once set, see the related expiry-logic finding), and an incident occurs on that exact gauge during that exact epoch, emergencyCouncil calling its emergency-cap function to halt emissions has no effect whatsoever. The release logic will still allow whatever the stale governance cap permits.

Location of Affected Code

File: [contracts/Voter.sol#L771-L845](#)

```
function _distributeEnforced(address _gauge, uint256 _epoch, uint256 _nominal, uint256 _gaugeLeft) internal {
    uint256 alreadyReleased = releasedByGaugeEpoch[_gauge][_epoch];
    uint256 requestedCumulative = alreadyReleased + _nominal;
    ControllerQuote memory quote = _safeQuoteGaugeCap[_gauge, _epoch, requestedCumulative, alreadyReleased];
    bool governanceCapActive = _governanceGaugeCapActive[_gauge, _epoch];
    bool reportUsable = quote.available && quote.status == REPORT_STATUS_VALID;
    uint256 policyAllowed;

    if (governanceCapActive) {
        policyAllowed = (requestedCumulative * governanceGaugeCapBps[_gauge][_epoch]) / MAX_BPS;
    } else if (reportUsable) {
        policyAllowed = quote.allowedCumulativeEmission;
    } else if (_capFallbackAllowed(quote)) {
        if (emergencyGaugeCapActive[_gauge][_epoch]) {
            policyAllowed = (requestedCumulative * emergencyGaugeCapBps[_gauge][_epoch]) / MAX_BPS;
        } else if (defaultGaugeCapActive[_gauge]) {
            policyAllowed = _fallbackPolicyAllowed(
                _gauge,
                _epoch,
                requestedCumulative,
                defaultGaugeCapBps[_gauge],
                quote.status,
                FALLBACK_REASON_DEFAULT_GAUGE_CAP
            );
        } else if (globalFallbackCapActive) {
            policyAllowed = _fallbackPolicyAllowed(
                _gauge,
                _epoch,
                requestedCumulative,
                globalFallbackCapBps,
                quote.status,
                FALLBACK_REASON_GLOBAL_FALLBACK_CAP
            );
        } else {
            policyAllowed = _fallbackPolicyAllowed(
                _gauge,
                _epoch,
                requestedCumulative,
```

```

    MAX_BPS,
    quote.status,
    FALLBACK_REASON_FAIL_OPEN_NOMINAL
  };
}
}

```

Impact

It won't let `emergencyCouncil` clear or override a governance cap entry, only the governor can write to that state.

Recommendation

Have the emergency cap always participate as an additional restriction rather than being skipped whenever a governance cap is active.

Team Response

Acknowledged.

I-01

● INFO

ACKNOWLEDGED

No Upper Bound on Emission Multiplier

Description

Both the global multiplier and per-gauge override only validate that the value is non-zero and different from the current value. There is no upper sanity bound. This multiplier scales directly into the emission cap formula in `quoteGaugeCap`. A single governor transaction can set this to an arbitrarily large value, instantly and effectively removing the emission cap for one gauge or the entire protocol. No timelock or multi-step delay is visible within this contract itself.

Location of Affected Code

File: [contracts/gauge-caps/GaugeCapController.sol#L178-L183](#)

```

function setMultiplierWad(uint256 _multiplierWad) external onlyGovernor {
    if (_multiplierWad == 0) revert InvalidReport();
    if (multiplierWad == _multiplierWad) revert SameValue();
    multiplierWad = _multiplierWad;
    emit GlobalMultiplierSet(_multiplierWad);
}

```

Recommendation

Add an explicit hard ceiling constant (e.g., `MAX_MULTIPLIER_WAD`) and enforce `require(_multiplierWad <= MAX_MULTIPLIER_WAD)` in both setters.

Team Response

Acknowledged.

I-02

● INFO

ACKNOWLEDGED

CLTwapOracle.sol and Gauge_pendingFees() Are Unreferenced Dead Code

Description

Two unrelated pieces of code in this codebase are fully defined but never invoked from anywhere:

`CLTwapOracle.sol` implements an on-chain Uniswap V3 TWAP-based price oracle (adapted TickMath/OracleLibrary logic). It is not imported by any other contract in scope, not referenced by `GaugeCapControllers.sol` (which is the contract that would plausibly need an on-chain price cross-check), and the only places it's mentioned anywhere in the repository are its own file and the attribution line in NOTICE.md. Verified via a repo-wide search: no other file imports or references it. This is notable because GaugeCapController's gauge-cap valuation is otherwise entirely trust-based (a governor-appointed publisher or signer-quorum submits values with no on-chain sanity check), `CLTwapOracle` looks like it was built specifically to close that gap and then never wired in.

`Gauge_pendingFees()` (contracts/gauges/Gauge.sol:106-130) is a defined internal function that computes pending fee amounts for the gauge's two underlying tokens. It is never called by any other function in `Gauge.sol`, and it's not exposed as an external/public function either, so there's no way to invoke it at all, internally or externally. Verified via a repo-wide search: `_pendingFees()` has no call sites anywhere in the codebase.

Location of Affected Code

File: [contracts/gauges/Gauge.sol#L106-L130](#)

```
function _pendingFees() internal view returns (uint256 pending0, uint256 pending1, address token0, address token1) {
    if (!isPool) {
        return [0, 0, address(0), address(0)];
    }
}
```

```
IPool pool = IPool(stakingToken);
[token0, token1] = pool.tokens();
pending0 = fees0 + pool.claimable0(address(this));
pending1 = fees1 + pool.claimable1(address(this));
```

```

uint256 supplied = IERC20(stakingToken).balanceOf(address(this));
if (supplied == 0) {
    return (pending0, pending1, token0, token1);
}

uint256 delta0 = pool.index0() - pool.supplyIndex0(address(this));
if (delta0 > 0) {
    pending0 += (supplied * delta0) / PRECISION;
}

uint256 delta1 = pool.index1() - pool.supplyIndex1(address(this));
if (delta1 > 0) {
    pending1 += (supplied * delta1) / PRECISION;
}
}

```

Recommendation

- If `CLTwapOracle` was meant to backstop the trust-based gauge-cap valuation pipeline (`GaugeCapController.publishGaugeEpochValue()`), wire it in as a sanity check before launch, or remove it if the integration was deprioritized, shipping it unused is misleading scaffolding that could be mistaken for an active safeguard.
- If `_pendingFees()` was meant to support a fee-accounting feature that didn't make it into this version, either finish wiring it in or remove it, an unreachable, unexposed function serves no purpose and adds audit surface for no benefit.

Team Response

Acknowledged.

I-03

● INFO

ACKNOWLEDGED

`Voter.reset()` **Authorizes Against Raw** `msg.sender`
Instead of `_msgSender()` **, Breaking Meta-tx Calls**

Description

Voter inherits ERC2771Context with a real trusted forwarder. Every other approval-gated function resolves the caller through `_msgSender()`: `vote()`, `depositManaged()`, `withdrawManaged()`, `claimBribes()`, `claimFees()`. `reset()` is the one exception. When called through the trusted forwarder, `msg.sender` inside the contract is the forwarder's address, not the end user, so `isApprovedOrOwner` checks the forwarder's own (nonexistent) approval and reverts.

Location of Affected Code

File: [contracts/Voter.sol#L308-L311](#)

```
function reset(uint256 _tokenId) external onlyNewEpoch(_tokenId) nonReentrant {
    if (!IVotingEscrow[ve].isApprovedOrOwner(msg.sender, _tokenId)) revert NotApprovedOrOwner(); //
    contracts/Voter.sol:309, raw msg.sender
    _reset(_tokenId);
}
```

Impact

When called through the trusted forwarder, `msg.sender` inside the contract is the forwarder's address, not the end user, so `isApprovedOrOwner()` checks the forwarder's own (nonexistent) approval and reverts.

Recommendation

Use `_msgSender()` instead of `msg.sender`.

Team Response

Acknowledged.

I-04

● INFO

FIXED

Unescaped Token Symbols in On-Chain SVG Allow XML Injection and Broken NFT Metadata

Description

The `quoteTokenSymbol` and `baseTokenSymbol` are sourced from an `ERC20's symbol()` call. They are used in the `NFTSVG::generateSVG()` function that calls the `generateTopText()` function to return the `svg`. The tokens are derived in `NonfungibleTokenPositionDescriptor::tokenURI()` function from the `positionManager.positions`. But the `positionManager` address is a user-supplied address and the function has no access control.

It is the same situation with `CLFactory::createPool()`, where anyone can create a pool with arbitrary tokens. This means that any attacker can deploy a token with a malicious `symbol()` return value and poison the on-chain metadata of every NFT minted against that pool.

Location of Affected Code

File: [contracts/periphery/libraries/NFTSVG.sol#L60-L86](#)

```
function generateTopText(
    string memory quoteTokenSymbol,
    string memory baseTokenSymbol,
    uint256 tokenId,
    int24 tickSpacing
```

```

} private pure returns (string memory svg) {
    string memory poolId =
        string(abi.encodePacked("CL", tickToString(tickSpacing), "-", quoteTokenSymbol, "/", baseTokenSymbol));
    string memory tokenIdStr = string(abi.encodePacked("ID #", tokenId.toString()));
    string memory id = string(abi.encodePacked(poolId, tokenIdStr));
    svg = string(
        abi.encodePacked(
            '<g id="',
            id,
            '">',
            '<text ...><tspan x="56" y="85.5938">',
            poolId,
            "</tspan></text>",
            ...
        )
    );
}

```

Impact

A token deployed with `symbol()` returning e.g. `"></g><script>..."` breaks out of the `id="..."` XML attribute and injects arbitrary markup, a symbol containing a bare `<` or `&` produces malformed XML that fails to render entirely. This permanently corrupts or hijacks the on-chain metadata for every NFT holder of that pool/gauge, not just the attacker, a griefing vector against any innocent LP who later stakes a position in that pool.

Proof of Concept

1. Attacker deploys an ERC20 with `symbol()` returning `X"><image href=x onerror=alert(1)//`.
2. Attacker creates a CL pool pairing this token with a legitimate token.
3. Any NFT minted on that pool calls `generateSVG`, which calls `generateTopText()`, embedding the hostile symbol unescaped into the `id` attribute and `<tspan>` text.
4. The resulting `tokenURI JSON/SVG` is malformed or contains injected markup, breaking rendering in marketplaces or executing injected content in any renderer that doesn't sandbox SVG.

Recommendation

Escape `"`, `'`, `<`, `>`, and `&` in every token-symbol derived string before interpolating it into SVG output (attribute or text-node context), e.g. via a byte-by-byte escaping helper analogous to Uniswap V3's `escapeQuotes`:

```

- string memory poolId =
-   string(abi.encodePacked("CL", tickToString(tickSpacing), "-", quoteTokenSymbol, "/", baseTokenSymbol));
+ string memory poolId =
+   string(abi.encodePacked("CL", tickToString(tickSpacing), "-", _escapeXML(quoteTokenSymbol), "/",
+     _escapeXML(baseTokenSymbol)));

```

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)